Qué es lo que ocurre cuando hacemos

```
django-admin startproject ProjectEx3
```

Django crea automáticamente la estructura base del proyecto web para poder comenzar a desarrollar. Vamos a verlo en detalle:

¿Por qué Django-admin?

Django-admin es el programa ejecutable global de Django. Cuando se instala Django con: `pip install django` dentro del entorno virtual (VENV recordemos que es el entorno virtual aislado para un proyecto) python también instala un script ejecutable llamado: `django-admin` ese script es una herramienta de **administración del framework.**

Es decir, literalmente es el administrador del framework. El cual sirve para:

- Crear proyectos
- Crear apps
- Ejecutar migraciones
- Iniciar el servidor
- Crear superusuarios
- etc…
-

`django-admin` es global, no sabe nada de tu proyecto específico hasta que se crea. Cuando se crea el proyecto con el comando de `django-admin startproject ProjectEx3` también se generan varios ficheros ya explicados, uno de ellos manage.py que internamente llama a `django-admin`, pero ya configurado con `DJANGO_SETTINGS_MODULE = 'ProjectEx3.settings'` por eso se utiliza `python manage.py runserver`

y no `django-admin runserver` porque manage.py sabe qué proyecto estás utilizando.

Django-admin puede ejecutar funciones internas de Django como son:

- modificar la base de datos
- crear estructuras de archivos
- levantar un servidor web
- crear registros en la base de datos (como superusuarios)

Todo eso ya está programado dentro del framework. Django-admin solo actúa como interfaz de línea de comandos.

Un proyecto es la estructura principal de nuestras aplicaciones web. Necesitamos hacer un sitio web con Django, entonces para esto necesitamos un contenedor que llevará la configuración global del sitio. Esta estructura principal se encarga aparte de la configuración global, las URLS principales, configuración de BD, ajustes de seguridad etc.. Se encarga también de integrar las demás aplicaciones del proyecto. Y con apps se refiere a que se pueden integrar varias aplicaciones dentro del proyecto principal.

Estas apps son módulos para nuestra aplicación web. Podríamos crear una app que sea por ejemplo Usuarios, y aquí incluir lo relacionado con usuarios, formularios, lógica de lo que puede realizar un usuario en un sitio web. Otra App puede ser estadísticas, una app que represente las estadísticas de los usuarios cuando se registren o naveguen. Se puede implementar aplicaciones dentro de lo que sería la app de usuario o relacionado. Esto no quiere decir que no existan o coexistan aplicaciones dentro del proyecto que sean distintas o de distinto enfoque.

La estructura conceptual de Django es de esta manera:

```
project/
 |
 ├── manage.py  → este es el punto de entradas del proyecto, se utiliza para
                  ejecutar comandos como python manage.py runserver o
                  python manage.py migrate o makemigrations.
 |
 ├── config/ →Aquí está la configuración global del sistema
             Configuración de la BBDD, middleware, apps instaladas,
             Seguridad, URLs raíz, Configuración ASGI/WSGI
 |   ├── __init__.py
 |   ├── asgi.py →Configura el servidor ASGI (estandar de producción)
 |   ├── wsgi.py → Interfaz clásica para servidores WSGI
 |   ├── urls.py →Es el router principal, define las rutas y conecta las URLS
de           cada App.
 |   └── settings/  → Divide la configuración por entorno
 |       ├── __init__.py
 |       ├── base.py →Esta es la configuración común, las apps instaladas,
                  middleware, configuración de la BD, la zona, idioma...
 |       ├── dev.py → Configuración de el entorno de desarrollo, BD local,
                 configuración más permisiva
 |       ├── prod.py → Configuración de producción, no hay debug, la
seguridad                 es estricta, la configuración real de la BD.
 |       └── test.py → Aquí se configura el tema del testing.
 |
 ├── apps/ → Esta carpeta contiene las aplicaciones del dominio. Cada App
             representa un módulo funcional independiente
 |   ├── usuario/
 |   ├── producto/
 |   └── pago/
 |
```

```
├── shared/
│
└── requirements.txt
```

Para crear una aplicación nueva se debe de utilizar el comando:

```
python manage.py startapp usuario
```

```
o también:
python manage.py startapp producto
```

```
o también:
python manage.py startapp usuario
```

cada carpeta que hay dentro de apps

```
├── apps/
│   ├── usuario/
│   ├── producto/
│   └── pago/
```

Es una app como tal reutilizable, con una funcionalidad específica. Este comando crea una estructura de carpetas y archivos de esta manera:

```
pago/
__init__.py
  admin.py →Registro del panel admin, gestor visual para poder manipular la
            BD
  apps.py →Configuración de la app
  migrations/ →Cambios en BD, cada vez que creemos modelos o hagamos
            cambios en ellos, django crea una serie de instrucciones para
            actualizar la BD. Sin tocar código SQL.
  models.py →Modelos de base de datos
  tests.py →Pruebas automáticas
  views.py →Lógica de negocio
```

Nos tenemos que mover al proyecto donde esté el manage.py para crear la app. Y poder ejecutar `python manage.py startapp pago`

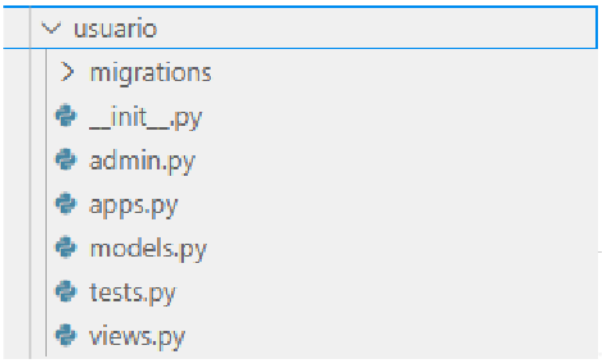
De esta manera podemos ver toda la estructura de ficheros y carpetas. Todas las aplicaciones se manejarán de la misma manera.

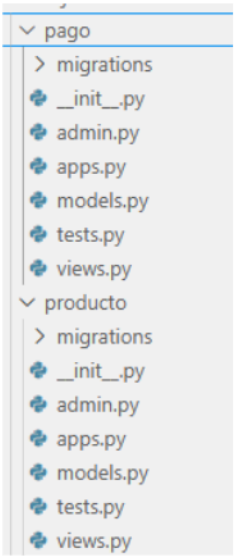
Se debe registrar la app en INSTALLED_APPS las aplicaciones que hemos creado dentro del settings.py del proyecto.

```
INSTALLED_APPS = [
    "apps.usuario",
    "apps.producto",
    "apps.pago",
```

```
]
```

Se añade coma al final para que no nos de error el archivo de configuración. De esta manera se relaciona la aplicación con el proyecto completo de django que tenemos creado.





Cada App debe contener solo la lógica relacionada con su responsabilidad.

La App de Usuario debería manejar:

- Login
- Logout
- Registro
- Perfil
- Recuperación de contraseña
- Edición de datos del usuario

Entonces esa lógica se maneja dentro de las vistas. Cada App debe tener vistas que se indican dentro del archivo views.py indicada dentro de cada app.

En Django las vistas son las funciones (clases o objetos) que:

- Reciben una petición HTTP (request)
- Procesa lógica
- Devuelve una respuesta (response)

Request → VISTA → Response

Una vista puede consultar una base de datos, validar formularios, autenticar usuarios, **renderizar plantillas HTML,** redirigir, manejar errores etc…

El ejemplo más real es este:

```
from django.shortcuts import render

# Create your views here.
#Vista de la página de login
def login(request):
    return render(request, 'usuario/login.html')
```

Primero se importa la función render (para renderizar una página html) este es un “atajo” que hace 3 cosas:

1. Cargar un template de HTML
2. Le pasa los datos (si hay)
3. Devuelve una respuesta en forma de HTTPResponse listo para enviar al navegador

Es decir:

Cuando alguien entra en la URL por ejemplo http://127.0.0.1:8000/ o localhost:8000

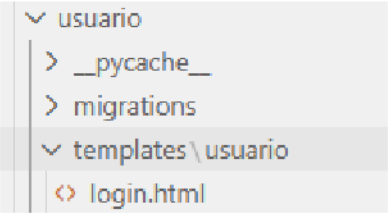
```
http://127.0.0.1:8000/usuario/login/
|     |        |    |
|     |        |    └── Ruta
|     |        └── Puerto
|     └── IP (tu máquina)
└── Protocolo
```

Si escribes http://127.0.0.1:8000/login

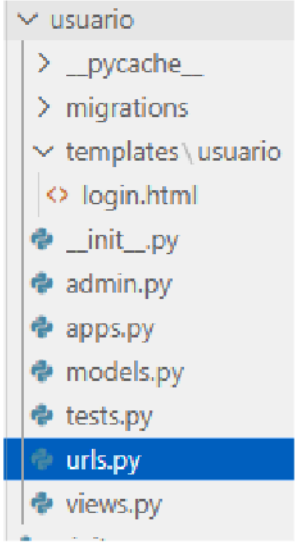
1. La petición va por http a 127.0.0.1 al puerto 9000 hacia la ruta /usuario/login
2. El servidor de desarrollo de Django (que corre en el runserver) recibe esa petición
3. Django busca en urls.py qué vista corresponde a esa ruta.
4. Ejecuta la función correspondiente en este caso login
   ```
   def login(request):
   ```
5. Esa función devuelve:
   ```
   return render(request, 'usuario/login.html')
   ```
6. Django construye el HTML y lo envía de vuelta al navegador
7. El navegador muestra la página

`usuario/login.html`

No está creado, por lo que hay que crearlo. Y debes crearlo en una carpeta llamada templates. En cada App hay que crear las carpetas templates/usuario, templates/pago, templates/producto y dentro los archivos a los que se está refiriendo la vista.



En las Apps creadas no viene por defecto un archivo para el enrutamiento por lo que hay que crearlo también. Solo viene en la aplicación principal del proyecto. Cada app debe ser independiente y reutilizable ya que si se quiere reutilizar solo la app producto en otro proyecto, debería llevarse también sus URLs.



(Enrutamiento)
Dentro de la App de usuario creamos el archivo urls.py que contiene este código:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.login, name='login'),
]
```

De esta manera le estamos diciendo a Django qué función (vista) debe ejecutarse cuando alguien entra a cierta URL.

1. Se utiliza `path` para definir las rutas, ej: path("ruta", funcion_que_se_ejecuta)
2. Desde la misma carpeta `from .` como se está dentro de usuarios/ursl.py entonces la carpeta es `.` ahí existe views.py, es lo mismo que decir import usuario.views
3. Django busca siempre una variable llamada urlpatterns, y esta variable debe ser una lista. Dentro de esa lista van todas las rutas de esta App.
4. `path('', views.login, name='login')` Cuando alguien entre a /login/ ejecuta views.login es decir ejecuta la función login. Si tuvieras una función logout dentro de la vista del usuario entonces habría que añadir el path nuevo: `path('logout/', views.logout, name='salir'),`

Cómo hemos metido en el proyecto una carpeta nueva para las apps llamada Apps, hay que incluir un archivo __init_.py para que pueda funcionar la aplicación.

Enrutamiento

En Django el enrutamiento es cómo el servidor sabe qué vista ejecutar según la URL que entra en el navegador.

USUARIO → NAVEGADOR → URL → urls.py principal → urls.py de la app → Template → HttpResponse →Navegador

Entonces hay que conectar las Apps al proyecto para que el proyecto sepa dónde se encuentra cada App. Es decir, al ser apps independientes, hace que el proyecto sea modular. En el urls.py del proyecto:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('Apps.usuario.urls')),
]
```

Se importa el panel de administración de Django que viene listo desde la instalación para utilizarlo. `admin.site.urls` son las rutas internas que Django usa para mostrar la interfaz de administración. Definimos las rutas URL con el path(). Como hay que delegar el manejo de rutas a otras apps hay que utilizar `include()`.
Cuando una petición HTTP llega, Django recorre la lista de arriba abajo para decidir qué vista ejecutar, esta lista es la que está en `urlpatterns = []`

Si alguien entra a http://127.0.0.1:8000/admin/ Django ejecuta las URLs internas del admin, esto es lo que permite ver y gestionar usuarios, modelos y datos desde el panel de administración. (Esta sección la veremos más adelante).

El path `path('', include('Apps.usuario.urls'))`, significa que todo lo que venga después de la raíz del proyecto se delega a `Apps.usuario.urls.`

Estáticos

Los archivos estáticos son recursos que el navegador necesita tal cual, sin que Django los modifique:

- CSS → Estilos
- Javascript → Scripts
- Imágenes, íconos, fuentes
- PDFs assets descargables

Cada app dentro del proyecto tiene sus propios estáticos, pero eso no significa que no puedan compartir entre sí archivos estáticos, por ello hay que evitar duplicidades, es decir, si un archivo estático (JS/CSS) se utiliza en varias apps, hay que ponerlo en una carpeta **global/shared.**

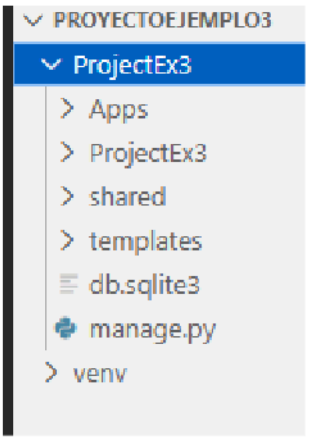
```
ProjectEx3/
├── apps/
│   ├── usuario/
│   │   ├── static/
│   │   │   └── usuario/
│   │   │       ├── css/
│   │   │       │   └── login.css
│   │   │       ├── js/
│   │   │       │   └── login.js
│   │   │       └── images/
│   │   │           └── avatar-default.png
│   │   └── templates/
│   │       └── usuario/
│   │           └── login.html
│   ├── producto/
│   │   └── static/producto/... (idem)
│   └── pago/
│       └── static/pago/... (idem)
│
├── shared/
│   └── static/
│       ├── css/
│       │   └── global.css
│       ├── js/
│       │   └── helpers.js
│       └── images/
│           └── logo.png
│
├── ProjectEx3/
│   └── settings/
│       └── base.py
```

Dentro de cada app la parte estática debe tener los archivos específicos de esa App. Y cuando hay recursos reutilizables hay que utilizar la carpeta compartida.

Esto no es todo ya que hay que añadir esta referencia de los estáticos al archivo de configuración global del proyecto en el settings.py

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/

STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'shared/static']
```



El `BASE_DIR` es el proyecto base, es decir ProjectEx3 y luego tenemos que indicar. El `BASE_DIR` apunta a la carpeta que contiene el manage.py en este caso PROYECTOEJEMPLO3/ProjectEx3

`STATIC_URL` indica la URL pública desde la cual se servirán los archivos estáticos. Siempre con / al final porque se deben formar de forma correcta los path de los estáticos.

Entonces metemos nuestra referencia del estático en la plantilla correspondiente, es este caso en usuario/templates/login.html

Para los archivos HTML, Django tiene su propio lenguaje de plantillas llamado **DLT** (Django Template Language). El cual permite insertar variables, lógica y loops dentro del HTML.

Ejemplo:

```
<p> Hola, {{usuario.username}}</p>
```

Si el usuario.username = “Carlos” se renderiza como:

```
<p> Hola, Carlos</p>
```

En este caso de los estáticos hay que utilizar DLT para establecer los estáticos dentro de la página HTML de esta manera:

```
{% load static %}
<!DOCTYPE html>
<html lang="es">
<head>
```

```
<link rel="stylesheet" href="{% static 'usuario/css/login.css'%}" >
<meta charset="UTF-8">
```

mediante {% load static %} se le dice a Django que quieres utilizar la librería de tags para archivos estáticos. Y que cargue la funcionalidad estática de los css: `{% static 'usuario/css/login.css'%}` sin esto no funcionará.

Modelado y BD

Hasta ahora hemos creado varias Apps dentro del proyecto:Usuario, Producto, Pedido, y Pago. Pero tener Apps no significa que el sistema esté modelado. Antes de escribir una sola línea en models.py, hay que entender cómo funciona el negocio y cómo se relacionan los datos.

Lo que hace Django en este caso sigue un orden:

1- Define los modelos
2- Se hace las migraciones
3- Se crean las tablas
4- Se usa ORM
5- Se construyen las vistas
6- Se construye la API y se interactúa con las templates

Un modelo en Django es la forma de representar y gestionar los datos del sistema. Es una clase de Python que **define cómo será una tabla de base de datos, qué columnas tendrá, qué relaciones existen con otras tablas y qué reglas básicas se aplican a esos datos.** Cuando se define el modelo en Django, no solo se guarda una estructura en memoria, se usa el modelo para generar migraciones, crear las tablas reales en la BD, validar datos y permitir consultar y modificar registros mediante el ORM sin escribir SQL a mano.

Tiene sentido utilizar la misma base de datos para desarrollo y producción donde sea posible, para evitar diferencias menores en el comportamiento.Utilizaremos la BD SQLite para los ejemplos ya que no esperamos unos requerimientos de accesos concurrentes elevados en la BD, otro plus es que no requiere trabajo adicional para ponerla en marcha.

En el fichero settings.py

```
# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Lo que tenemos aquí es una configuración predeterminada que utiliza la BD SQLite3. Django trabaja con BD SQL relacionales y es compatible con más motores de bases de datos. Bases de datos compatibles con Django

para inicializar la BD hay que ejecutar a la altura de donde se encuentra el manage.py:

```
python manage.py migrate
```

```
(.venv) → ProjectEx3 python manage.py migrate

Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

Después de entender las relaciones y decisiones de integridad, el siguiente paso es convertir esas decisiones en modelos Django reales, y justo después, crear físicamente las tablas con migraciones.

Django trabaja con:

- Cada clase que hereda de models.Model será una tabla;

Las migraciones son el sistema que utiliza Django para propagar los cambios que haces en el modelo, por ejemplo, añadir un campo, eliminar una tabla, cambiar un nombre… hacia el esquema de la BD.

models.Model es la clase base que DJango usa para convertir una clase de Python en una entidad persistente en Base de datos. Cuando una clase hereda de models.Model, Django la registra como "modelo", inspecciona sus campos (CharField, EmailField, ForeignKey etc.) y con esa información puede generar migraciones y crear tablas reales.
Si no hereda de models.Model, sería una clase normal de Python, no hay tabla, no hay ORM, no hay migraciones, no hay admin, no hay .objects.

Cada App contiene los modelos que son responsabilidad de ese módulo. No se crea una carpeta "models" global para todo el proyecto, de ahí que exista un models.py por cada App, y un makemigrations que trabaja por cada app.

Es decir:

- Los modelos de usuario en usuario/models.py
- Los modelos de pago en pago/models.py
- Los modelos de producto en producto/models.py

¿Si apareciera un nuevo concepto como Pedido?

¿Cómo se haría el modelo dentro de Producto?

```python
from django.db import models

# Create your models here.
class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField()
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField()

    def __str__(self):
        return self.nombre
```

Se crea la clase producto dentro de producto/models.py y se introduce las columnas con sus tipos de datos. Estos tipos de datos pueden ser desde ChaField, TextField, DecimalField, IntegerField… son tipos de columna. Se limita el nombre del producto a un máximo de 100 caracteres para optimizar. Se le indica que el precio debe ser decimal con un número máximo de 10 dígitos y que en la parte decimal tenga 2 dígitos.

Para decirle a Django que debe de leer el archivo de models que tiene la App producto, que tenemos esa clase nueva y que cree la tabla nueva debemos ejecutar:

```
python manage.py makemigrations
```

```
(.venv) →  ProjectEx3 python manage.py makemigrations
Migrations for 'producto':
  Apps/producto/migrations/0001_initial.py
    + Create model Producto
```

Esto genera los archivos de migración necesarios para que la base de datos pueda adaptarse a esos modelos. Esto solo crea o actualiza ficheros dentro de cada app, en migrations/, que describen los cambios del esquema que hay que aplicar.

Compara 2 cosas, el estado actual de los modelos y el último estado que quedó registrado en la migración anterior. Si detecta diferencias como puede ser por ejemplo, se ha creado un modelo nuevo, se ha añadido un nuevo campo, se ha cambiado un tipo, se ha añadido una relación nueva…) genera un archivo tipo 0001_initial.py, 0002_add_stock.py etc.. Dentro de ese archivo hay operaciones como CreateModel, AddField, AlterField, RemoveField, AddConstraint… Pero aún no se ejecuta, solo se hacen constancia de los cambios.

Para aplicar las migraciones se debe utilizar:

`python manage.py migrate`

```
● (.venv) →  ProjectEx3 python manage.py migrate
  Operations to perform:
    Apply all migrations: admin, auth, contenttypes, producto, sessions
  Running migrations:
    Applying producto.0001_initial... OK
```

Esto aplica a las migraciones a la base de datos. Es el momento en el que Django ejecuta SQL y la BD cambia. Aquí Django asegura que las tablas internas existen, migraciones, auth, sesiones etc. Después crea o modifica las tablas según las migraciones.

En este punto se puede validar el modelo si está bien creado antes de continuar.

Con:
`python manage.py showmigrations`

```
pago
 (no migrations)
producto
 [X] 0001_initial
sessions
 [X] 0001_initial
usuario
 (no migrations)
```

Se observa qué está marcada la migración como aplicada.

Una vez existe la estructura de BD, la validación también se puede dar desde el panel de administración de Django ("admin"), este panel es una aplicación web que Django trae ya construida para gestionar datos del sistema. Django lo incluye porque necesita consultar, crear, editar, borrar registros de BD y construir esa interfaz a mano ralentiza el desarrollo. El admin se genera automáticamente a partir de los modelos, por eso es tán útil justo después de migrar. Si los modelos están bien construidos, el admin entiende las tablas y las relaciones, posteriormente muestra en forma de formularios.

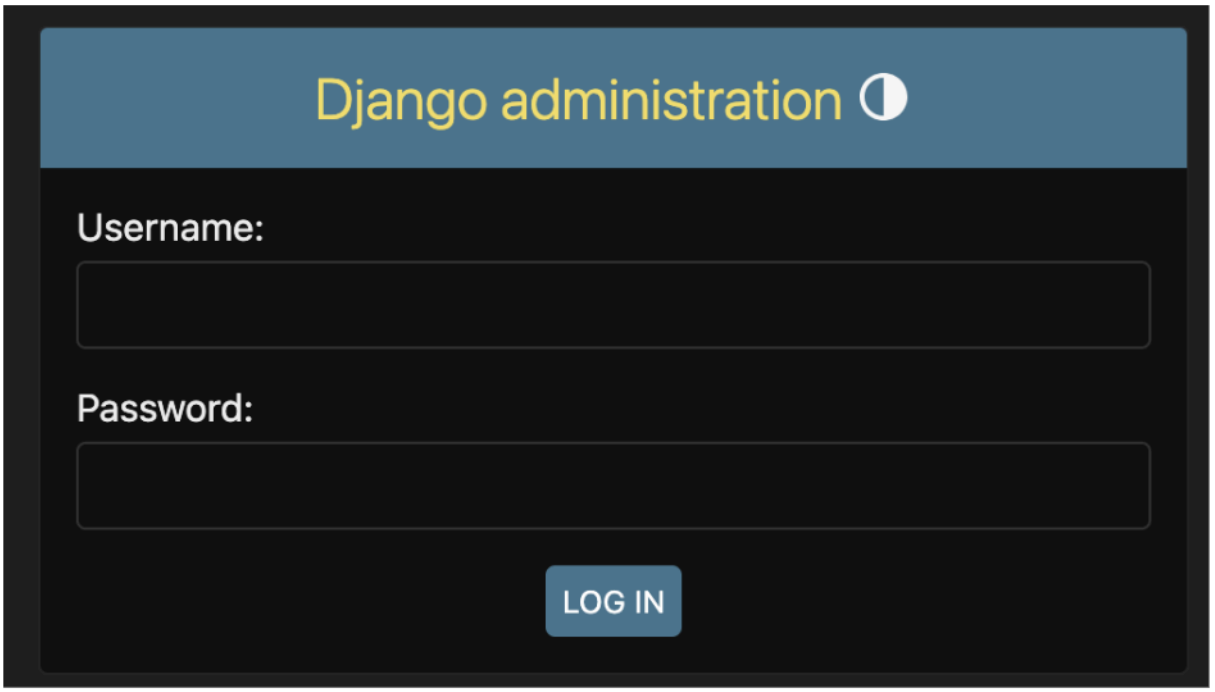
Para esto hay que registrar el modelo de la app en su producto/admin.py

```
from django.contrib import admin

# Register your models here.
from .models import Producto

admin.site.register(Producto) #Aquí
```

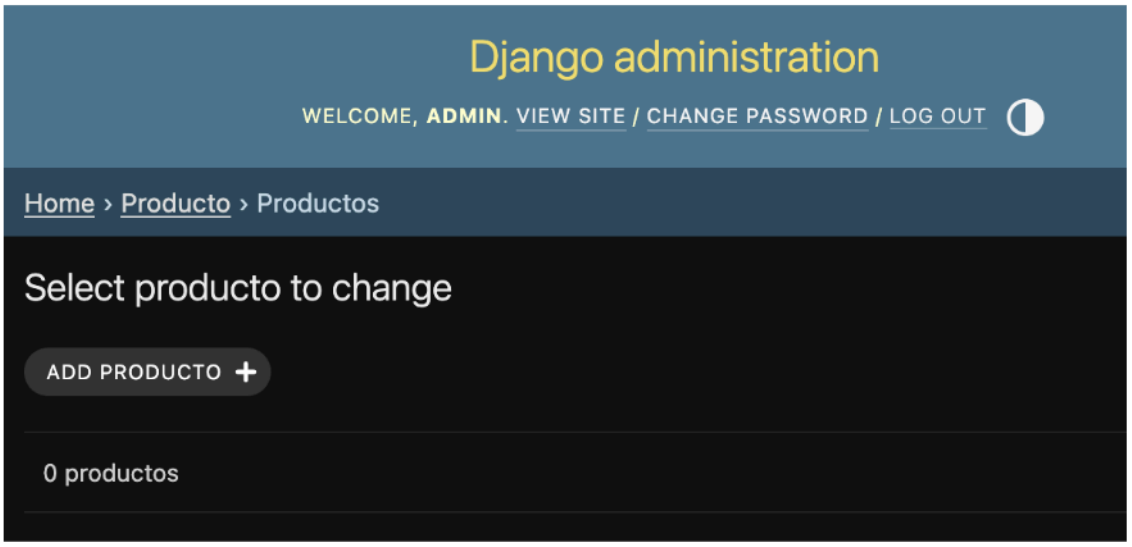
Para poder acceder a http://localhost:8000/admin/login/?next=/admin/

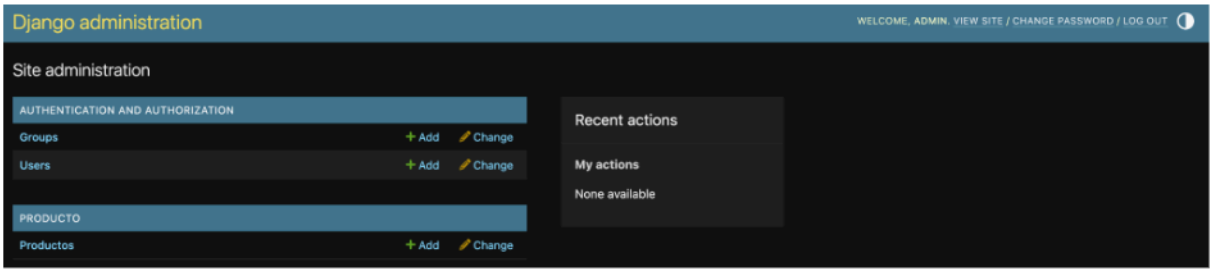


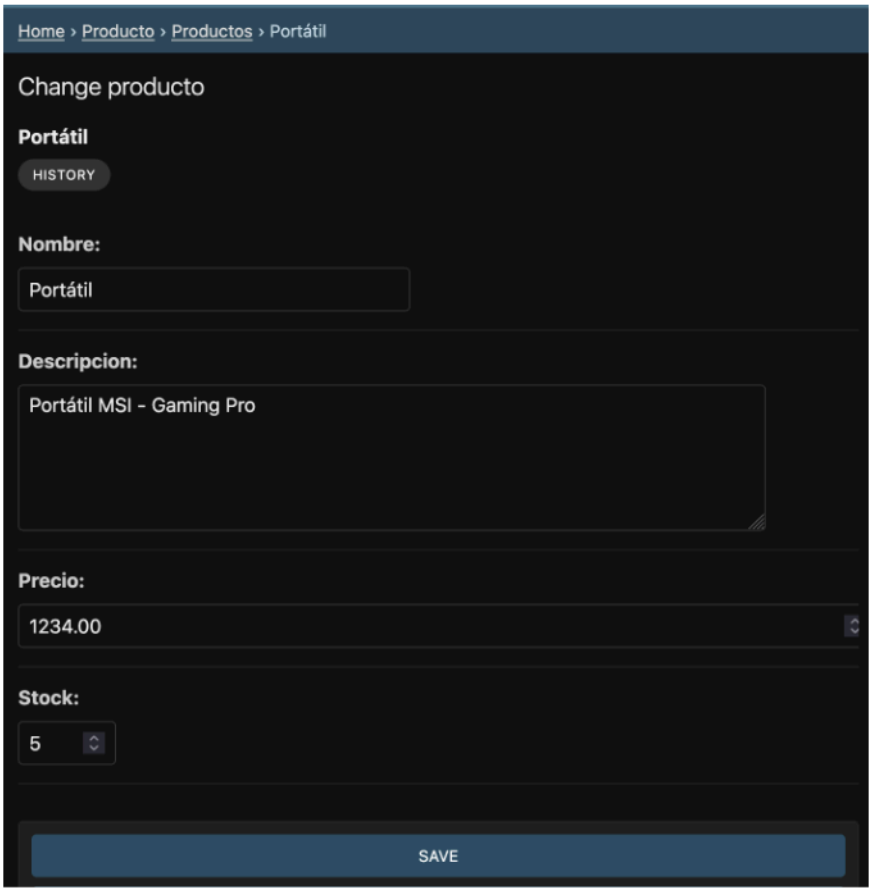
Hay que tener permisos de superusuario ya que es una cuenta con permisos totales (root).
Por lo tanto hay que crear un superusuario:
python3 manage.py createsuperuser

```
(.venv) →  ProjectEx3 python3 manage.py createsuperuser
Username (leave blank to use 'maui'): admin
Email address: mcalixto@egibide.org
Password:
Password (again):
```

Entramos en producto y podemos crear uno nuevo.







Se puede modificar, borrar, y añadir un nuevo producto.

También se puede cargar muchos productos desde un archivo JSON con el comando:

python3 manage.py loaddata productos.json

el archivo json tiene una estructura de bloque como esta:

```
{
  "model": "producto.producto",
  "pk": 1,
  "fields": {
    "nombre": "Portátil Gaming Pro",
    "descripcion": "RTX 4060, 16GB RAM",
    "precio": "1199.99",
    "stock": 10
  }
}
```

`"model": "producto.producto"` es el identificador único para django y se divide en 2 partes:

- **`producto`** (antes del punto): Es el nombre de tu **App** (la carpeta).
- **`producto`** (después del punto): Es el nombre de la **Clase** (el modelo) que escribiste en `models.py`.

`"pk": 1`

Significa **Primary Key** (Clave Primaria).

- **Regla de oro:** No puede haber dos productos con la misma `pk`. Si intentas cargar un JSON con una `pk` que ya existe, Django sobrescribirá el producto anterior con los datos nuevos.

`"fields": { ... }`

Aquí dentro van los datos reales que se han definido en el modelo:

- **Formato:** Los nombres de las claves (`nombre`, `precio`, etc.) tienen que ser **idénticos** a como están en el `models.py`.
- **Tipos de datos:** * Los textos van entre comillas (`" "`).
    - Los números decimales (como el precio) van entre comillas en el JSON de Django para evitar errores de precisión.
    - Los números enteros (`stock`) pueden ir sin comillas.

Una vez importado los productos hay que mostrarlos. En Django se hace siempre con el mismo flujo: una URL llama a una vista, la vista consulta datos con el ORM (NO UTILIZAS SQL) y devuelve una respuesta HTML renderizando una plantilla: URL → VISTA → ORM → Plantilla → HTML

Por lo que hay que crear una nueva definición de una función nueva para listar los productos y que estos se puedan ver en la plantilla. (**Recuerda**) La vista es el punto donde se controla qué se muestra y con qué datos. No es el HTML, no es la BD, es la capa intermedia que recibe la petición del navegador (request), ejecuta la lógica (ej: Leer productos) y devuelve una respuesta (response).

En este caso la lógica es simple: pedir a Django todos los productos que existen en la BD. Eso se hace con el ORM usando el modelo Producto, que representa la tabla producto.

Dame todos los productos: **Producto.objects.all()** esa consulta ORM de Django significa dame todos los registros (todas las filas) de la tabla producto.

La vista con la función de listarProductos quedaría de la siguiente manera:

```
from django.shortcuts import render
from .models import Producto
# Create your views here.
#Vista de la página de productos
def listarProductos(request):
    productos = Producto.objects.all()
    return render(request, 'producto/producto.html', {"productos": productos})
```

Y la plantilla debe de modificarse también:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Tienda de Productos</title>
</head>
<body>
    <h1>Catálogo de Productos</h1>
    <table border="1">
        <thead>
            <tr>
                <th>Nombre</th>
                <th>Descripción</th>
                <th>Precio</th>
                <th>Stock</th>
            </tr>
        </thead>
        <tbody>
            {% for p in productos %}
            <tr>
                <td>{{ p.nombre }}</td>
                <td>{{ p.descripcion }}</td>
                <td>{{ p.precio }} €</td>
                <td>{{ p.stock }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</body>
</html>
```

De esta manera accediendo a http://localhost:8000/producto/ obtendremos: el catalogo de productos:

Catálogo de Productos

Nombre	Descripción	Precio	Stock
Portátil Gaming Pro	RTX 4060, 16GB RAM, 512GB SSD	1199.99 €	10
Ratón Inalámbrico	Ergonómico con carga USB-C	25.50 €	150
Teclado Mecánico	Switches Blue, Retroiluminado RGB	79.00 €	45
Monitor 4K 27"	Panel IPS, 144Hz para diseño	349.99 €	20
Auriculares Bluetooth	Cancelación de ruido activa	129.00 €	60
Alfombrilla XL	Superficie de microfibra 90x40cm	15.90 €	200
Webcam 1080p	Ideal para streaming y reuniones	45.00 €	35
Silla Ergonómica	Soporte lumbar y reposacabezas	189.50 €	12
Micrófono USB	Patrón cardioide para podcasting	88.00 €	25
Disco Duro 2TB	Externo, USB 3.0 ultra rápido	65.00 €	80
Smartphone 5G	Pantalla OLED 6.7 pulgadas	599.00 €	30
Funda de Cuero	Para móvil de última generación	19.99 €	100
Cargador Rápido 65W	GaN tecnología, carga 3 dispositivos	39.90 €	55
Tableta Gráfica	8192 niveles de presión	150.00 €	18
Altavoz Inteligente	Sonido 360 y control por voz	49.00 €	90
Router WiFi 6	Doble banda, alta velocidad	115.00 €	22
Smartwatch Sport	GPS integrado y sensor cardíaco	210.00 €	40
Cámara de Seguridad	Visión nocturna y alertas móvil	34.50 €	75
Hub USB-C 7 en 1	HDMI 4K, SD, USB 3.0	29.00 €	110
Lámpara de Escritorio	LED ajustable y carga inalámbrica	42.00 €	28
Mando Consola	Compatible con PC y consolas	59.99 €	50
Soporte Portátil	Aluminio ventilado y plegable	24.00 €	65
Memoria RAM 16GB	DDR4 3200MHz (Kit de 2)	68.50 €	40
SSD M.2 1TB	Velocidad lectura hasta 3500MB/s	95.00 €	33

Relaciones

En una base de datos relacional no conectas objetos, conectas tablas mediante claves. Eso significa que si una tabla necesita apuntar a otra, guarda un identificador que referencia una fila existente. En Django esas conexiones se expresan con 3 mecanismos:

- OneToOneField para 1:1 Uno a uno
- ForeignKey para 1:N
- ManyToManyField para N:N